Journal of King Saud University Computer and Information Sciences (2026) 38:825
https://doi.org/10.1007/s44443-026-01248-7

RESEARCH

Multi-objective reinforcement learning scheduling framework for heterogeneous AI chips: Energy-efficient and latency-aware edge-cloud collaboration

Shijia Shao[1] · Peiqing Ye[2] · Biao Zhao[3] · Hao Lu[4]

Received: 9 May 2026 / Accepted: 12 August 2026
© The Author(s) 2026

Abstract
Scheduling AI inference across heterogeneous edge-cloud chips requires balancing energy, latency, cost, and thermal feasibility. This study presents a chip-aware scheduling framework, rather than a new multi-objective reinforcement learning theory. A 53-dimensional state describes directed-acyclic-graph tasks, CPU/GPU/NPU/FPGA status, network conditions, and queue slack. A fuzzy controller adjusts energy, latency, and cost priorities, while a hybrid-action Proximal Policy Optimization policy jointly selects the offloading target, physical chip, and dynamic-voltage-and-frequency-scaling coefficient. Dependency, deadline, thermal, and bandwidth constraints are enforced through action masks and residual penalties. In traffic-video and industrial-inspection simulations, the framework achieved 5.8 TOPS/W, 132 ms average latency, a normalized cost coefficient of 0.17 per task, and 85.2% Pareto coverage. Under an equal 1000-episode and 30-seed budget, PPO reached 95% of its asymptotic improvement in 580 ± 45 episodes, obtained a final normalized return of -0.15 ± 0.03 and an HV of 0.87 ± 0.02, and produced no divergent seed. EdgeCloudSim/iFogSim2 replication and hardware-in-the-loop testing preserved the advantage over DTRL; in hardware-in-the-loop tests, the framework required 31.5 ± 3.5 J per task and 142 ± 8.0 ms, compared with 35.8 ± 4.0 J and 155 ± 9.5 ms for DTRL. The results indicate that task-aware objective adaptation and chip-level control can reduce wasteful offloading and thermal stress while preserving service timeliness.

Keywords Heterogeneous AI chips · Edge-cloud collaboration · Multi-objective reinforcement learning · Dynamic voltage and frequency scaling · Thermal-aware scheduling · Sustainable computing

1 Introduction

Artificial intelligence (AI) services are increasingly deployed across edge devices and cloud data centers. This edge-cloud structure can reduce response time by processing urgent tasks near the data source while using cloud accelerators for compute-intensive subtasks (Luo and Dai 2024; Li and Wu 2023). However, heterogeneous AI chips such as CPUs, GPUs, NPUs and FPGAs differ substantially in computing capability, power consumption, thermal threshold, programming overhead and network access cost. A scheduler that only minimizes latency may overuse high-power cloud GPUs, whereas a scheduler that only minimizes energy may delay safety–critical tasks (Bai et al. 2024; Li et al. 2025). Therefore, the central problem of this manuscript is how to make real-time scheduling decisions that jointly reduce energy consumption, maintain deadline satisfaction, control chip temperature and avoid unnecessary resource cost in heterogeneous edge-cloud collaboration (Abbache, 2025; Rajesh and Pyne 2022).

✉ Shijia Shao
576683457@163.com

Peiqing Ye
492644546@163.com

Biao Zhao
879018711@163.com

Hao Lu
850426123@163.com

[1] School of Software, Northwestern Polytechnical University, Xi'an, China

[2] School of Mechanical Engineering, Tsinghua University, Beijing, China

[3] Shanghai Six-united Electronics, Shanghai, China

[4] Shanghai Yunyi Electronics, Shanghai, China

Published online: 31 August 2026

Existing scheduling methods do not fully solve this problem. Static hardware-performance rules cannot adapt to sudden workload changes or bandwidth fluctuation. Single-objective deep reinforcement learning (DRL) can generate online decisions, but it usually optimizes one target and therefore hides the conflict among energy, latency and cost (Gu et al. 2025; Naik and Alam, 2025; Ma and Hou 2025; Zhang, 2025). Adaptive MORL and preference-conditioned policies improve objective weighting, but many of them focus on preference exploration or computation offloading and do not explicitly combine task type, chip-level thermal-frequency dynamics, and link bandwidth in one action space (Han et al. 2025; Pournazar et al., 2025; Basaklar et al. 2023; Miao et al. 2025). This gap is important for sustainable computing because unnecessary offloading, overheating and repeated migration increase energy use and may shorten device life.

To address the above gap, the proposed framework answers three research questions. First, how can task descriptors, heterogeneous chip states and network states be represented as a compact online scheduling state? Second, how can the scheduler adapt the relative priority of energy, latency and cost according to task urgency and hardware conditions without relying on fixed manual weights? Third, how can a reinforcement learning policy jointly decide offloading location, chip selection and DVFS-level frequency adjustment while respecting deadline, thermal and bandwidth constraints? The answers to these questions define the technical contribution of the manuscript and avoid presenting the work as a general MORL algorithm.

The contribution is an engineered scheduling framework with four jointly implemented components, not a new MORL theory. (1) A chip-aware 53-dimensional state model integrates workload size, DAG dependency, utilization, temperature, normalized resource price, bandwidth, and queue slack. (2) A fuzzy controller converts task urgency, energy sensitivity, and cost sensitivity into normalized objective weights. (3) A hybrid-action PPO policy jointly selects the offloading target, the physical CPU/GPU/NPU/FPGA node, and the DVFS coefficient, while dependency, deadline, thermal, and bandwidth masks remove infeasible choices. (4) A completed verification protocol combines matched-budget actor-critic comparison, simulator sanity checks, multi-objective reference-set construction, network/hardware robustness tests, EdgeCloudSim/iFogSim2 replication, hardware-in-the-loop measurements, and resource-pricing sensitivity. The technical novelty therefore lies in the coordinated chip-aware scheduling design and its constraint-handling integration, rather than in claiming a new general-purpose scalarization or reinforcement-learning principle. Figure 1 summarizes this route from problem definition to empirical verification.

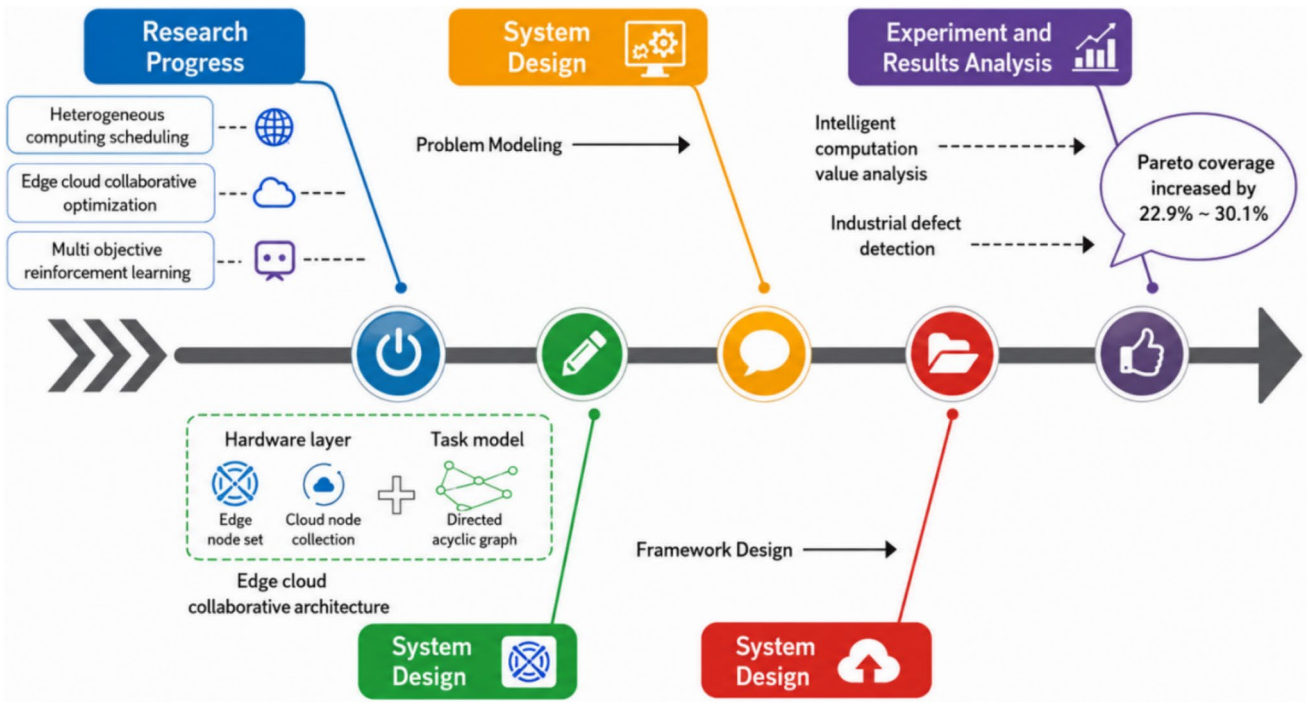


Fig. 1 System Architecture Diagram

2 Research progress on heterogeneous computing scheduling and multi-objective optimization

2.1 Heterogeneous computing scheduling

Heterogeneous computing scheduling aims to coordinate the efficient collaboration of processors with different architectures (such as CPU, GPU, NPU) to meet the demands of diverse computing tasks (Kazemi et al., 2025). Early research mainly relied on static strategies, such as pre-defined rules based on hardware performance databases to allocate computationally intensive tasks to GPUs and low-power tasks to CPUs. Although this type of method can simplify the scheduling process, it is difficult to adapt to dynamic load changes and hardware state fluctuations (Nagabushnam and Kim 2025; Zheng et al. 2025). In recent years, dynamic deep reinforcement learning (DRL) has gradually become a research hotspot, which dynamically adjusts task allocation strategies by sensing real-time chip computing power, temperature, and energy consumption data. For example, some scholars have proposed scheduling algorithms based on Q-learning, which generate real-time decisions based on task computation and current chip load, achieving a 20% reduction in latency in image processing scenarios (Jiang et al., 2025). However, existing DRL methods often focus on a single optimization objective, such as minimizing latency or energy consumption, and lack the global balancing ability for multi-objective conflicts, leading to resource imbalance in edge-cloud collaboration scenarios (Karishma and Kumar 2025; Saad et al., 2025).

2.2 Edge-cloud collaborative optimization

The core of edge-cloud collaborative optimization lies in balancing the benefits of local computing and remote offloading, and its research path can be divided into two categories: energy efficiency priority and latency priority (Peng et al. 2024). Energy efficiency priority methods typically reduce system energy consumption through task compression, computation offloading, and hardware sleep mechanisms, such as migrating high-energy model training tasks to the cloud and performing only lightweight inference at the edge (Cheng et al. 2025; Dui et al. 2024; Yang et al., 2024). This type of method shows significant performance in improving the battery life of IoT devices, but may lead to a surge in communication latency due to excessive reliance on the cloud (Zeng et al. 2024). The delay priority strategy emphasizes real-time performance, such as prioritizing the allocation of critical tasks to edge nodes in autonomous driving scenarios, but its high computing power requirements may lead to overheating or excessive energy consumption of edge devices (Du et al. 2023). Recent studies have attempted to integrate two types of strategies, such as threshold based dynamic switching mechanisms, but these methods rely on manual parameter tuning and are difficult to adapt to complex and changing network environments and task loads, and have not fully considered the collaborative potential of heterogeneous chips (Yang et al. 2023).

2.3 Multi-objective reinforcement learning

Multi-objective reinforcement learning provides a new approach for complex decision-making problems by simultaneously optimizing multiple competitive objectives (Nguyen et al. 2025). Traditional methods often use linear scalarization, such as weighting delay, energy consumption and cost into a single reward value, but their weight settings rely on prior knowledge and are difficult to adapt to dynamic scenes (Jang et al. 2025; Parhizi et al. 2025). In contrast, Pareto-frontier-based distributed reinforcement learning can generate a set of non-dominated solutions for decision makers to choose according to their preferences. For example, some studies use evolutionary algorithms to generate Pareto-optimal solution sets and then use reinforcement learning to select dynamic strategies (Wang et al. 2025). However, such methods have high computational complexity and have not fully solved the nonlinear coupling among objectives. In edge-cloud collaborative scenarios, network bandwidth fluctuations and hardware heterogeneity further increase the difficulty of multi-objective optimization, and existing methods still have a significant gap between real-time responsiveness and solution-set diversity (Zheng et al. 2025; Han et al. 2025).

2.4 Positioning relative to adaptive MORL and heterogeneous scheduling

Recent adaptive MORL studies have introduced preference vectors, decision-tree policies, adaptive scalarization and transfer-based offloading. The proposed framework is therefore positioned more narrowly as a heterogeneous AI-chip scheduler for edge-cloud inference workloads, not as a new general-purpose MORL theory. Its distinction lies in the combination of four elements in one online loop: task-type fuzzy inference, explicit NPU/GPU/FPGA/CPU hardware profiles, thermal-frequency prediction under DVFS, and bandwidth-aware offloading. Existing adaptive MORL methods usually adapt preferences or offloading decisions, whereas this framework also selects the physical chip and the frequency level and then evaluates feasibility

through deadline, thermal and bandwidth masks (Han et al. 2025; Pournazar et al., 2025; Basaklar et al. 2023; Miao et al. 2025). For self-contained notation, the first mentions of related algorithms and indicators are expanded as Non-dominated Sorting Genetic Algorithm II (NSGA-II), Non-dominated Sorting Genetic Algorithm III (NSGA-III), Multi-Objective Evolutionary Algorithm based on Decomposition (MOEA/D), hypervolume (HV), and inverted generational distance (IGD) (Table 1).

2.5 SDN/NFV-enabled green networking and cloud load balancing

Green cloud multimedia networking extends energy-aware resource management from compute servers to network functions and forwarding devices. NFV virtualizes functions that would otherwise be tied to dedicated appliances, while SDN exposes a programmable, logically centralized view of traffic paths and link states. The GreenVoIP framework, for example, coordinated virtualized service functions, servers, and switches to reduce the number of active devices while maintaining service quality (Montazerolghaem et al. 2020). This line of work is directly relevant to the sustainability motivation of the present study because communication energy, path selection, and network-device activation can offset the gains obtained from chip-level DVFS and accelerator selection.

Research on task scheduling and load balancing in SDN-based cloud computing similarly uses controller visibility of topology, link utilization, and server load to allocate tasks, minimize response time, and improve resource utilization. The recent review by Mahdizadeh et al. organizes rule-based, metaheuristic, and reinforcement-learning approaches and emphasizes the interaction between task scheduling, load balancing, and network-resource optimization (Mahdizadeh et al. 2025). Compared with those studies, the current environment does not implement an SDN controller or virtual-network-function placement; it treats residual bandwidth, round-trip time, and packet loss as observed network states.

The two research directions are complementary rather than competing. A cross-layer extension could expose SDN path energy, congestion, and VNF-placement state to the chip-aware scheduler and could add route or service-chain selection to the action vector. Such an extension would permit joint optimization of network paths, compute-node selection, and chip frequency. The present manuscript limits its claims to compute-side scheduling and identifies SDN/NFV-aware orchestration as a future integration path.

Figure 1 is used as a graphical roadmap of the manuscript. It starts from the sustainability-oriented scheduling conflict, proceeds to the state-action-reward design and PPO-based online decision process, and ends with statistical evaluation,

Table 1 Positioning of the proposed framework against related adaptive optimization methods

Aspect	Representative existing approach	Limitation for this problem	Clarification in this manuscript
Adaptive MORL preference handling	Preference-vector, decision-tree or transfer-based MORL (Han et al. 2025; Basaklar et al. 2023)	Adapts objective preference but usually does not model NPU/GPU/FPGA thermal thresholds and DVFS states together	Fuzzy weights are driven by task urgency, energy sensitivity, cost sensitivity, chip temperature and bandwidth state
Energy-centric offloading	Fog or edge offloading with an energy objective (Pournazar et al., 2025)	Often decides local/cloud execution but treats hardware as an abstract resource pool	The action space jointly covers offloading, chip selection and frequency regulation
Evolutionary multi-objective scheduling	NSGA-II, NSGA-III and MOEA/D-style Pareto search (Zhu et al. 2026)	Provides solution-set diversity but has limited online responsiveness under fast workload and bandwidth changes	The PPO policy outputs online decisions and uses Pareto coverage, HV and IGD for evaluation
Heterogeneous chip scheduling	Static and dynamic heterogeneous-chip scheduling surveys (Miao et al. 2025)	Requires clearer notation, task templates and validation for reproducibility	The revision adds notation units, simulator validation, statistical tests, and completed appendix analyses

ablation, constraint-violation analysis and sensitivity tables. This organization separates background knowledge, methodological choices and empirical evidence more clearly than the previous version.

3 Design of multi-objective reinforcement learning framework for edge-cloud collaboration

3.1 Edge-cloud collaborative architecture

The edge-cloud collaborative architecture provides infrastructure support for efficient execution of complex tasks by integrating the low-latency response capabilities of edge devices with the high-performance computing power of the cloud. However, the dynamic characteristics of heterogeneous hardware resources make it difficult for traditional static modeling methods to accurately describe the system state, and there is an urgent need for a dynamically aware architecture model (Constantin et al. 2023).

The hardware layer contains two edge NPUs, two edge CPUs, two cloud GPUs and two cloud FPGAs. Let $V = V_{\text{edge}} \cup V_{\text{cloud}}$ denote the set of available heterogeneous nodes and let $T = t_i$ denote the set of directed acyclic graph (DAG) subtasks. For node v_j, f_j denotes the available compute capability, P_j^{comp} denotes computing power, P_j^{net} denotes transmission power, θ_j denotes the current temperature, $\theta_j^{\max}$ denotes the safe thermal threshold, B_j denotes available bandwidth and C_j denotes the unit resource price. These symbols are used consistently in Formulas (1–12) and Table 2.

The task model is represented by a DAG $G = (T, E_{\text{DAG}})$, where E_{DAG} is the dependency-edge set and is no longer denoted by the same symbol as the edge-node set. A dependency edge (t_k, t_i) means that subtask t_i can start only after predecessor t_k has completed and the required data have been transferred. Each subtask is defined by workload W_i, input data I_i, output data O_i, deadline d_i and task type. This notation avoids the previous ambiguity in which the same symbol was used for edge nodes, DAG edges and the PPO clipping coefficient.

As shown in Fig. 2, the proposed dynamic parametric model constructs a refined description of the edge-cloud collaborative architecture, providing accurate system state inputs for subsequent multi-objective optimization. This model not only covers real-time changes in hardware computing power and energy consumption, but also quantifies the impact of temperature on computational efficiency, laying a theoretical foundation for the design of dynamic scheduling strategies.

3.2 Modeling of multi-objective optimization problems

In the edge-cloud collaborative scenario, task scheduling needs to simultaneously optimize three competitive objectives: energy consumption, latency and cost. The conflicting nature of these objectives makes traditional single-objective optimization methods unsuitable for direct application. Existing research often uses linear weighting to transform multiple objectives into a single-objective, but weight

Table 2 Notation and units used in the optimization model

Symbol	Meaning	Unit/domain	Used in
$T = t_i$	Set of DAG subtasks	index set	Task model and constraints
$V = v_j$	Set of heterogeneous edge/cloud nodes	index set	Scheduling decision
x_{ij}	Binary assignment of task t_i to node v_j	{0,1}	Energy, latency and cost models
c_i	Computational load of task t_i	GOP or normalized workload	Computation time
f_j	Effective compute rate of node v_j after DVFS scaling	GOP/s (TOPS or TFLOPS after conversion)	DVFS-scaled execution time
P_j^{comp}	Computing power of node v_j	W	Computational energy
P_j^{net}	Network transmission power of node v_j	W	Communication energy
B_j	Available bandwidth of node v_j	MB/s	Communication delay and bandwidth constraint
d_{ik}	Data transferred from predecessor t_k to successor t_i	MB	DAG communication cost
$tau_i^{start}, tau_i^{finish}$	Start and finish time of task t_i	ms	Dependency constraint
$theta_j$	Current chip temperature	degrees Celsius	Thermal constraint
$theta_j^{max}$	Safe temperature threshold	degrees Celsius	Thermal constraint
rho_j	Normalized unit-price coefficient of node v_j	dimensionless, [0,1]	Normalized cost model
w_E, w_L, w_C	Adaptive weights of energy, latency and cost	non-negative, sum to 1	Reward function

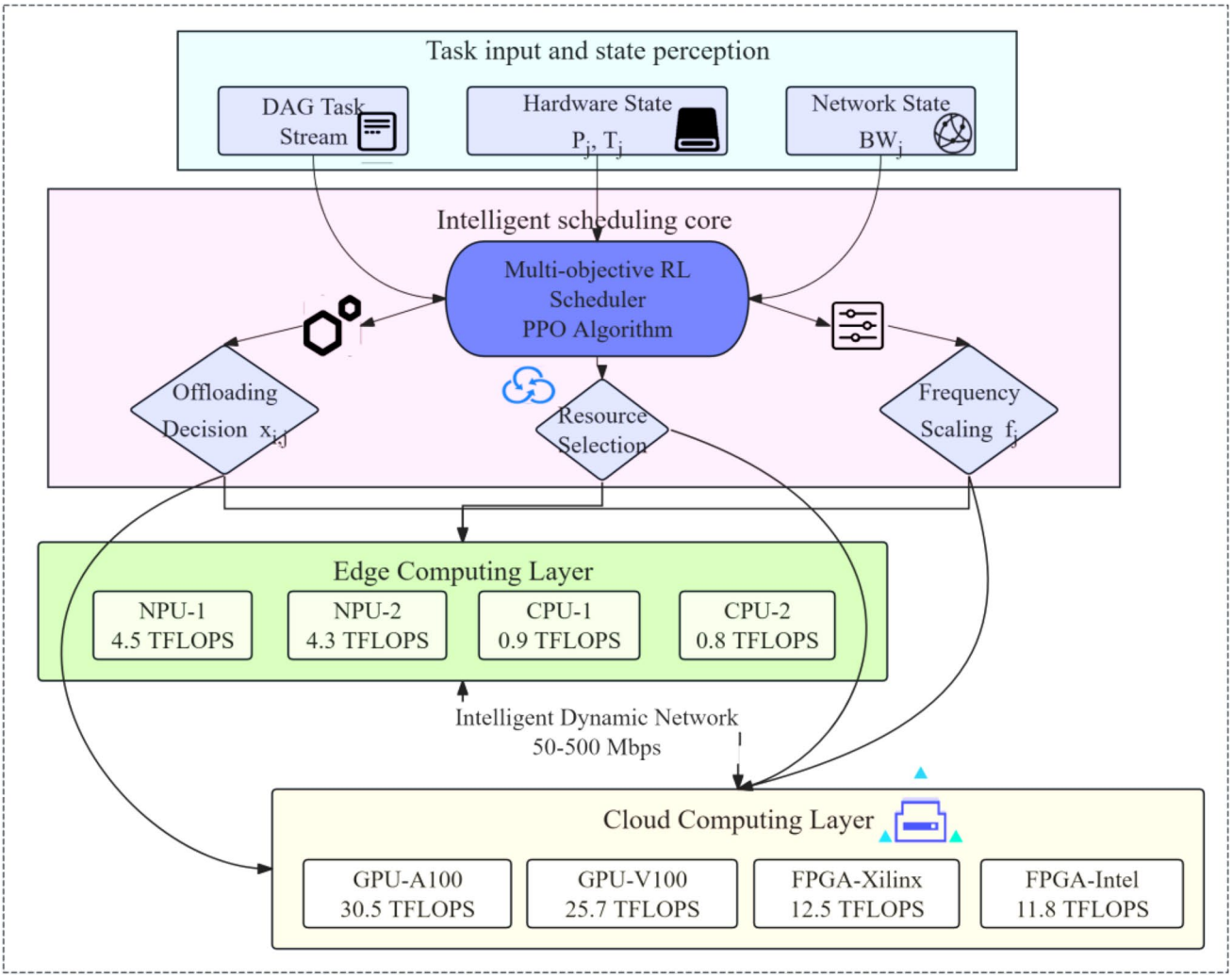


Fig. 2 Edge-cloud collaborative architecture diagram

selection relies on manual experience and is difficult to adapt to dynamic scene requirements.

In the optimization model, total energy consists of computational energy on the selected chip and communication energy across the selected link. Computational energy is calculated from workload, selected frequency and chip power, while communication energy is proportional to transferred data, transmission time and network power. Total latency is the critical-path completion time of the DAG, including execution time and inter-node transmission time. Total cost is the accumulated resource price multiplied by residence time on each selected node. These objective functions make the energy-latency-cost trade-off explicit and prepare the reward terms used by the PPO policy (Fig. 3).

To improve mathematical clarity, the variables used in Formulas (1-12) are defined before the objective functions. Energy is reported in joules, latency in milliseconds, bandwidth in MB/s, temperature in degrees Celsius, and cost as

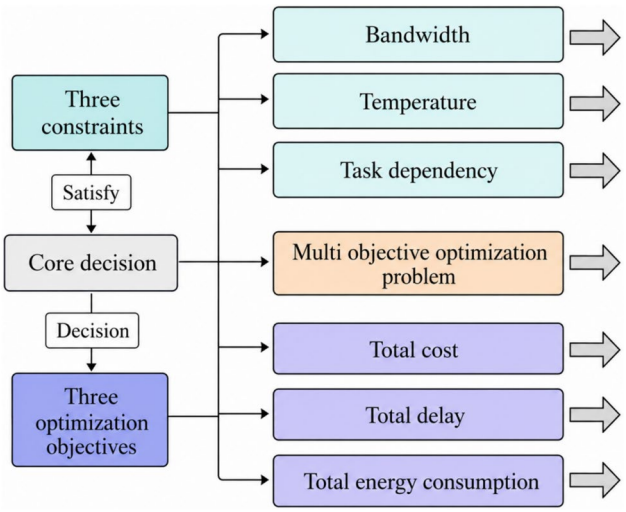


Fig. 3 Optimization problem flowchart

a dimensionless normalized coefficient per task. Computational workload is expressed in GOP, and the node term used in execution-time equations is the effective compute rate in GOP/s after DVFS scaling. Hardware peak capability may be reported in TOPS or TFLOPS for description, but clock frequency in GHz is not substituted directly into the workload-to-time ratio.

The optimization model minimizes the total energy consumption, total delay and total cost of the system, which are defined as follows:

Energy consumption model:

$$E_{total} = \sum_{i=1}^{K} \sum_{j=1}^{M+N} x_{ij} \cdot \left(\frac{D_i}{F_j} \cdot P_j + \frac{S_i^{in} + S_i^{out}}{B_j} \cdot P_j^{net} \right) \tag{1}$$

In Formula (1), $x_{i,j}$ indicates whether task t_i is assigned to node v_j, W_i is the computational workload of task t_i, f_j is the available computing frequency or capability of node v_j, P_j^{comp} is the computing power, $D_{k,i}$ is the inter-task transfer size, B_j is the available bandwidth and P_j^{net} is the network transmission power. The delay model is defined as follows:

For unit consistency, the symbol f_j in the execution-time terms is interpreted throughout as an effective compute rate, not as a raw clock frequency. If F_j^{nom} is the nominal hardware capability and $alpha_j$ is the selected DVFS coefficient, then $f_j = alpha_j F_j^{nom}$ after converting the capability to GOP/s. This definition makes c_i/f_j a time quantity and preserves comparability between CPU, GPU, NPU, and FPGA nodes.

$$T_{total} = \max_j \left(\sum_{i=1}^{K} x_{ij} \cdot \left(\frac{D_i}{F_j} + \frac{S_i^{in} + S_i^{out}}{B_j} \right) \right) \tag{2}$$

The task delay is composed of computation time and cross node communication time, and must meet critical path constraints. Cost Model:

$$C_{total} = \sum_{j=1}^{M+N} \left(a_j \cdot \sum_{i=1}^{K} x_{ij} \cdot \frac{D_i}{F_j} \right) \tag{3}$$

In Formula (3), C_j is the unit resource price of node v_j and the summation is taken over scheduled tasks and selected nodes. The following constraints define feasibility. The dependency constraint requires the start time s_i of each subtask t_i to be no earlier than the finish time e_k of every predecessor t_k plus its data-transfer time. The temperature constraint requires the predicted temperature θ_j to remain below $\theta_j^{\max}$ after frequency adjustment. The bandwidth constraint requires the instantaneous traffic assigned to node v_j to remain below B_j. These constraints are used first as hard action masks and then as soft reward penalties when a stochastic action produces a small residual violation.

Task dependency constraints:

$$t_{start}(v_i) \geq \max_{v_k \in pre(v_i)} \left(t_{end}(v_k) + \frac{S_{ki}^{out}}{B_{kj}} \right) \quad v_j \in v_E \cup v_C \tag{4}$$

Formula (4) states that the start time v_i of task v_k must be later than the completion time of every predecessor task plus the required data-transmission time. The temperature constraint is defined as follows:

$$T_j(t) \leq T_j^{\max}, \forall j \in \varepsilon \cup C \tag{5}$$

Formula (5) requires the predicted node temperature $T_j(t)$ to remain below the hardware safety threshold $T_j^{\max}$ for every selected chip. The bandwidth constraint is defined as follows:

$$\sum_{i=1}^{K} x_{ij} \cdot \left(S_i^{in} + S_i^{out} \right) \leq B_j^{\max}, \forall j \in \varepsilon \cup C \tag{6}$$

Formula (6) requires the total input and output traffic assigned to node $\forall j$ to remain within its available bandwidth $B_j^{\max}$. Together, Formulas (1-6) define the coupled energy-latency-cost optimization problem and its feasibility region. This formulation avoids treating heterogeneous edge-cloud resources as an abstract homogeneous pool and provides a mathematical basis for chip-aware sustainable scheduling.

For Formula (1), the first term denotes computational energy, calculated from task workload, assigned node frequency and chip power; the second term denotes communication energy, calculated from transferred data, available bandwidth and network power. Formula (2) contains both execution delay W_i/f_j and inter-node transmission delay $D_k, i/B_j$ when dependent subtasks are assigned to different nodes. Formula (3) multiplies node resource price by task residence time, so cloud GPU/FPGA resources are penalized when their latency benefit is insufficient. Formulas (4)-(6) are handled as hard constraints during action masking and as soft penalties in the reward when the sampled action violates dependency, thermal or bandwidth limits.

Before scalarization, each minimization objective q in {E, L, C} is normalized against scenario-level bounds as $q_{tilde} = (q - q_{min})/(q_{max} - q_{min} + 10^{-12})$. The PPO reward is the negative weighted sum of the three normalized objectives minus the constraint penalty: $r = -(w_E E_{tilde} + w_L L_{tilde} + w_C C_{tilde}) - lambda_v V$. The fuzzy controller supplies non-negative weights that sum to one, while V aggregates residual deadline, thermal, and bandwidth violations after hard action masking. Thus, Formulas (1-3) define the physical objectives, Formulas (4-6) define the feasible region, and Formula (9) supplies the learning signal used to optimize feasible scheduling behavior.

The reported cost is not a universal monetary billing claim. Let pi_j be an observed provider price or a locally defined resource charge. The simulator uses $rho_j = (pi_j - pi_{min})/(pi_{max} - pi_{min} + 10^{-12})$, so the base-case cost values are dimensionless coefficients per task. In Appendix Table A12, hourly provider or private-edge charges are first converted to per-task charges using measured resource residence time; computation and egress charges are combined before min–max normalization, and the scheduler is rerun under each price vector. The reported shifts therefore measure policy sensitivity to alternative price structures rather than retrospective rescaling of the final cost column.

For deadline feasibility, the estimated finish time is computed as: $t_{\text{current}} + \frac{W_i}{f_j} + \frac{I_i}{B_j} + \frac{O_i}{B_j} + q_j$ A candidate node is feasible only if: CP_i^{remain} where CP_i^{remain} is the remaining critical-path slack. For thermal feasibility, the one-step temperature prediction is: $\beta_j(\theta_{j,t} - \theta_{\text{amb}})\Delta t$ with: $P_j^{\text{idle}} + \kappa_j f_j^3$ under DVFS. These equations explain how deadline slack and post-frequency temperature are computed before action masking.

3.3 Design of Multi-Objective Reinforcement Learning Scheduling Framework

Multi-objective reinforcement learning provides a data-driven solution for complex scheduling problems by autonomously exploring the mapping relationship between environmental states and actions. However, traditional deep reinforcement learning methods face challenges such as high-dimensional state space, difficulty in discretizing action space, and multi-objective conflicts in heterogeneous scenarios.

3.3.1 Overall architecture

The state space integrates four categories: task descriptor, node status, network descriptor and queue descriptor. The task descriptor includes workload, input size, output size, deadline, task type and DAG level; the node status includes compute capability, utilization, power, temperature and unit cost for each of the eight nodes; the network descriptor includes bandwidth, round-trip time (RTT) and packet-loss indicator; and the queue descriptor includes ready-queue length, critical-path slack and remaining DAG ratio. The action space is hybrid: one discrete offloading target, one discrete chip index and one bounded continuous DVFS coefficient. This design is necessary because sustainable scheduling must decide not only where a task is executed but also which chip executes it and at what frequency (Fig. 4).

State space:

$$s_t \begin{cases} \text{Task attributes}: D_i, S_i^{in}, S_i^{out} \\ \text{Chip status}: F_j, P_j, T_j \\ \text{Network status}: B_j, Latency_j \end{cases} \tag{7}$$

Action space:

$$a_t = (o_t, j_t, \varphi_t), o_t \in \{edge, cloud\}, j_t \in \nu(o_t), \varphi_t \in [0.4, 1.0] \tag{8}$$

In Formula (8), o_t denotes the offloading target, which can be edge-side or cloud-side execution; j_t denotes the selected physical chip node from the feasible node set $\nu(o_t)$; and φ_t denotes the continuous DVFS frequency-scaling coefficient within [0.4,1.0]. This hybrid action enables the scheduler to jointly determine the execution location, chip selection and frequency level.

For each scheduling step, the state vector is concatenated from task descriptors, node status descriptors, network descriptors and global queue descriptors. In the experimental configuration with eight heterogeneous nodes and two scenarios, the resulting state dimension is $6+8\times5+4+3=53$. The action is a hybrid action composed of one discrete offloading target, one discrete chip index, and one continuous frequency scaling coefficient in [0.4, 1.0].

The parameters in Table 3 are not manually assigned arbitrary values. They are determined from four sources. The task descriptors are extracted from the DAG workflow templates used in the two experimental scenarios, including workload, input/output data size, deadline, task type and DAG level. The node-status descriptors are obtained from the heterogeneous hardware profile used in the simulator, including computing capability, utilization, power, temperature and unit cost. The network descriptors are generated from the bandwidth, RTT and packet-loss settings described in the experimental setup. The queue descriptors, including ready-queue length, critical-path slack and remaining DAG ratio, are computed online during scheduling. Therefore, Table 3 defines the policy input and action components, while the numerical configuration of hardware, network and workload parameters is reported in the experimental setup and supplementary materials.

The dynamic weight reward function is designed as follows:

$$R = w_E \cdot \left(\frac{E_{base} - E_{total}}{E_{base}}\right) + w_T \cdot \left(\frac{T_{base} - T_{total}}{T_{base}}\right) - w_C \cdot \left(\frac{C_{total}}{C_{base}}\right) \tag{9}$$

In Formula (9),w_E, w_T, w_C denote the adaptive weights for energy, latency and cost, respectively. These weights are generated by the fuzzy logic controller according to task urgency, energy sensitivity, cost sensitivity and hardware

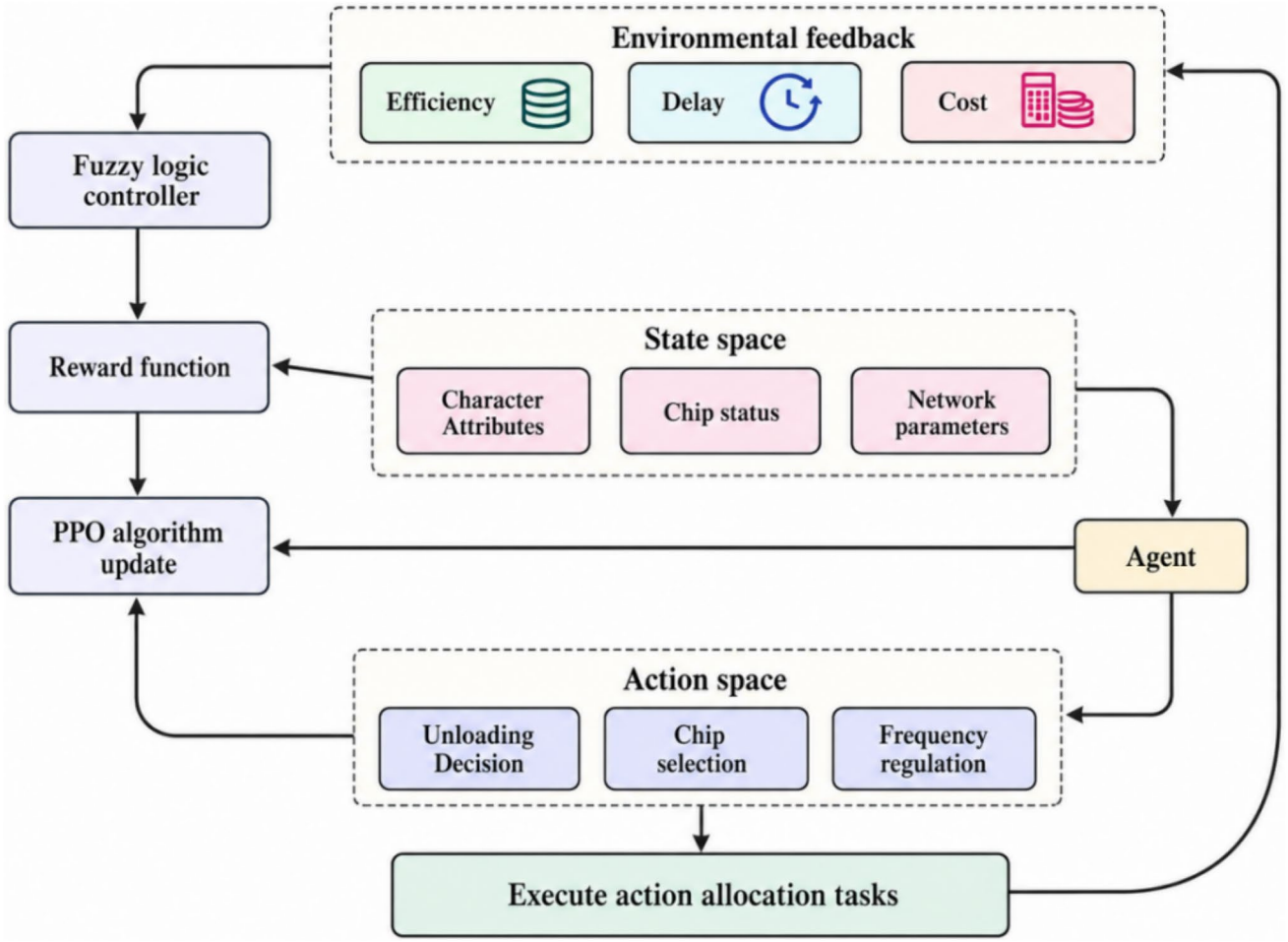


Fig. 4 Overall Architecture Diagram

Table 3 Concrete state and action design

Component	Variables included	Type	Dimension	Implementation note
Task descriptor	workload, input size, output size, deadline, task type, DAG level	continuous/categorical	6	Task type is one-hot encoded before policy input
Node status	compute capability, utilization, power, temperature, unit cost	continuous	$8 \times 5 = 40$	Eight nodes are used in Tables 2–4
Network descriptor	edge bandwidth, cloud bandwidth, RTT, packet-loss indicator	continuous	4	Bandwidth is sampled from the ranges in Table 2
Queue descriptor	ready-queue length, critical-path slack, remaining DAG ratio	continuous	3	Used to avoid local decisions that break DAG deadlines
Action component 1	edge/cloud/offloading target	discrete	1	Invalid targets are masked by dependency and bandwidth checks
Action component 2	chip selection among CPU/NPU/GPU/FPGA nodes	discrete	1	Temperature-infeasible chips are masked
Action component 3	frequency scaling coefficient	continuous	1	Mapped to the allowed DVFS interval of the selected chip

state, and $E_{base}, T_{base}, C_{base}$ denote the corresponding scenario-level benchmark values used for normalization.

The scalar reward used by PPO is $r_t = -\left[\omega_E(t)\frac{E_t}{E_{ref}} + \omega_L(t)\frac{L_t}{L_{ref}} + \omega_c(t)\frac{C_t}{C_{ref}}\right] - \lambda_v V_t$, where E_{ref}, L_{ref}, and C_{ref} are scenario-level benchmark values used for normalization, V_t is the aggregated violation score, and $\lambda_v = 2.0$ is the penalty coefficient for constraint violations.

The normalization constants are estimated from the training split only and are then fixed during validation and testing to prevent information leakage. The penalty coefficient $lambda_v$ is selected on the validation seeds so that one unit of normalized violation produces a larger loss than the largest single-objective improvement available from an infeasible action. This ordering ensures that adaptive weights trade off feasible schedules rather than rewarding systematic constraint violation Table 4.

3.3.2 Algorithm implementation

PPO is selected as an engineering choice after comparing the scheduling requirements with established actor-critic alternatives. PPO uses a clipped on-policy surrogate objective and can incorporate discrete action masks directly (Schulman et al., 2017). SAC is off-policy and entropy-regularized, which can improve sample efficiency but requires a carefully defined categorical-continuous adapter and entropy treatment for masked actions (Haarnoja et al. 2018). TD3 reduces critic overestimation for continuous control through twin critics and delayed updates (Fujimoto et al. 2018), while DDPG provides a deterministic continuous-action baseline (Lillicrap et al. 2016); both require additional wrappers for the two categorical decisions in this problem. This rationale motivates PPO but does not by itself establish empirical superiority.

Adopting the proximal strategy optimization algorithm, the objective function is:

$$J(\theta) = E_t\left[\min\left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}A_t, clip\left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}, 1-\varepsilon, 1+\varepsilon\right)A_t\right)\right] \tag{10}$$

In Formula (10), A_t denotes the advantage estimate, epsilon denotes the PPO clipping coefficient, π_θ is the updated policy and $\pi_{\theta_{\text{old}}}$ is the behavior policy used to collect the trajectory. The notation is standardized here so that epsilon is used only for PPO clipping and is not confused with edge sets or directed acyclic graph (DAG) dependency symbols.

NPU operator acceleration is retained as an implementation-level efficiency indicator. The speedup term is used to update effective task execution time when convolution operators are mapped from CPU execution to NPU execution. It is not used as an independent optimization objective. It affects energy and latency through W_i/f_j in the environment update.

To account for implementation-level acceleration on NPUs, we define an operator-mapping speedup for tasks containing convolution operators supported by the NPU runtime. Equation (11) estimates the execution-time reduction obtained when eligible convolution operators are mapped from CPU execution to NPU execution. This relation is used only to update the effective task duration in the scheduling environment and is not introduced as an additional optimization objective.

$$Speedup = \frac{T_{CPU}}{T_{NPU}} = \frac{F_{NPU}\cdot\eta_{NPU}}{F_{CPU}} \tag{11}$$

Migration from a near-edge node to a cloud GPU or FPGA may reduce pure execution time, but it can also introduce model conversion, code generation, compilation, and runtime-initialization overhead. To represent this platform-dependent penalty, Eq. (12) defines cross-platform conversion efficiency as the fraction of the end-to-end candidate task time that remains after deducting the conversion overhead. This quantity is an implementation-level

Table 4 Fuzzy logic rules for adaptive weight generation

Input condition	Fuzzy rule	Weight output	Constraint-handling action
high urgency + low slack	prioritize deadline satisfaction	(w_E,w_L,w_C) = (0.20,0.70,0.10)	mask slow cloud links and high-RTT offloading
high energy sensitivity + moderate slack	prioritize low-power execution	(w_E,w_L,w_C) = (0.60,0.25,0.15)	prefer NPU/CPU nodes below temperature threshold
large input data + congested bandwidth	penalize communication overhead	increase w_C and λ_v V_t	avoid cross-domain migration unless compute gain is large
high chip temperature	reduce thermal pressure	increase penalty term λ_v V_t	lower DVFS coefficient or mask overheated chip
low urgency + high cloud price	prioritize resource economy	(w_E,w_L,w_C) = (0.35,0.20,0.45)	select cost-efficient FPGA/CPU execution

efficiency indicator rather than an independent optimization objective.

$$Conversion_Efficiency = 1 - \frac{Overhead_{trans}}{Total_Time} \tag{12}$$

The resulting duration is then used to recalculate the energy, latency, and residence-time-based cost terms before the reward is evaluated. The cloud GPU or FPGA candidate is compared with the corresponding near-edge candidate using the same three-objective reward model. A low conversion-efficiency value indicates that conversion overhead offsets a substantial part of the accelerator benefit and therefore makes cross-platform migration less likely to be selected. Equation (12) does not introduce a separate reward term or a fourth optimization objective.

The actor and critic share a two-layer feature encoder and then use separate output heads. The actor has a categorical head for offloading, a categorical head for chip selection and a Gaussian head for the frequency-scaling coefficient. The critic outputs the scalar state value. Constraint handling is implemented in two stages: infeasible discrete actions are masked before sampling, and residual violations caused by stochastic frequency selection are added to V_t. The remaining 1.2% deadline and 2.1% bandwidth violations in Table 10 mainly arise from stochastic exploration and one-step prediction error near deadline or bandwidth boundaries, rather than from deliberately accepting infeasible schedules.

The proposed multi-objective reinforcement learning framework addresses multi-objective conflicts in heterogeneous scenarios through dynamic weighting mechanisms and refined state action design. This framework not only supports real-time policy generation, but also adapts to the optimization requirements of different task types, providing a scalable solution for edge-cloud collaborative scheduling (Fig. 5).

The program flow is presented before the pseudo-code-style steps. Figure 5 shows data loading, state construction, action masking, hybrid-action sampling, environment update, reward calculation and PPO parameter update. Supplementary Table S2 then summarizes the executable

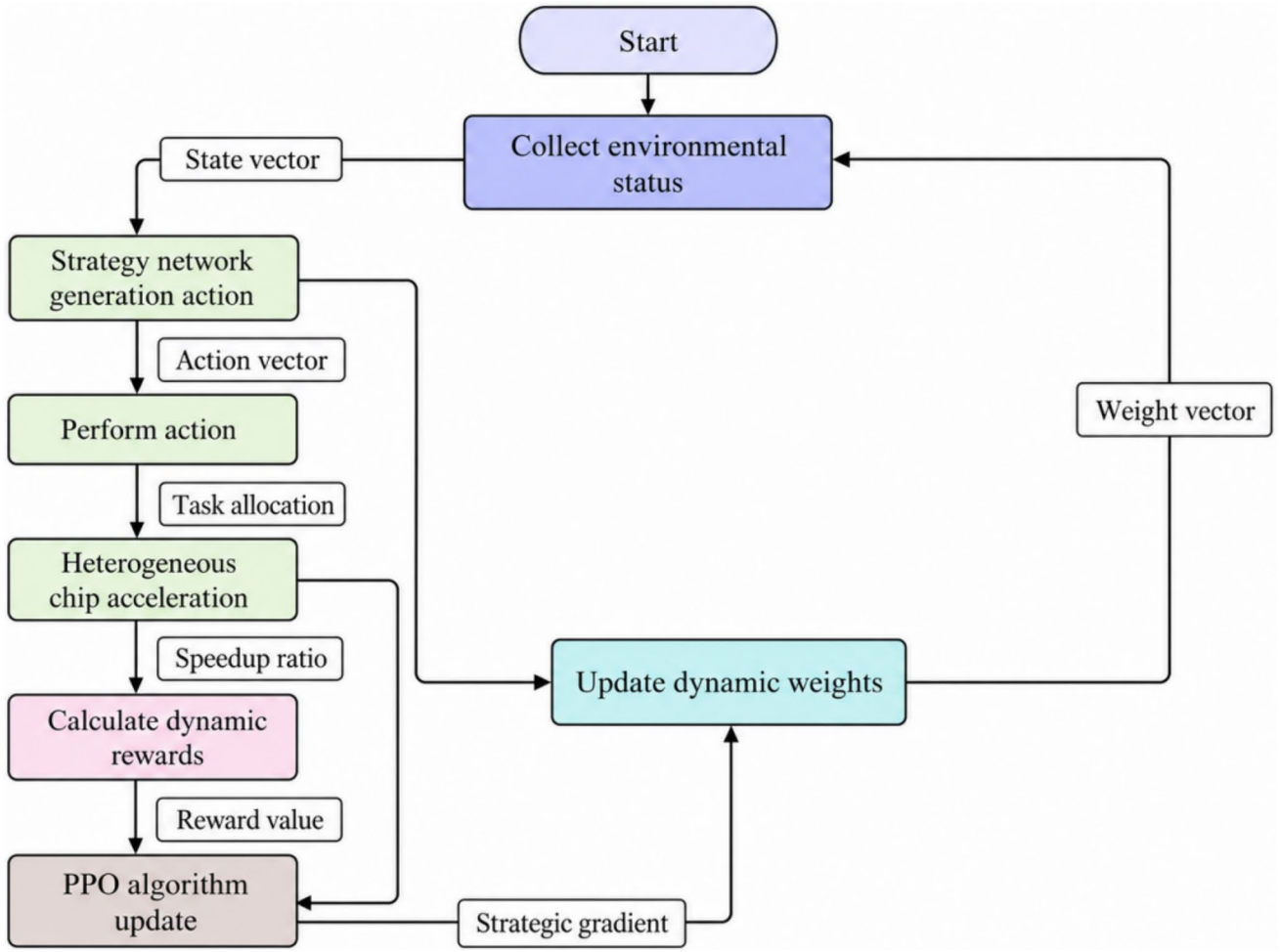


Fig. 5 Algorithm Implementation Process

algorithm steps so that readers can follow the transition from system flow to implementation details.

For reproducibility, each scheduling episode starts by sampling a DAG workflow, node utilization vector, bandwidth vector and initial temperature vector. The environment then executes the selected action, updates queue states and chip temperature, and returns the multi-objective reward. All baselines reuse the same random seeds, workload traces and bandwidth traces so that differences originate from the scheduling policy rather than task generation randomness.

The constraint-handling sequence is: (1) construct the ready-task set from DAG dependencies; (2) estimate $\widehat{e}i, j$ for each candidate node and mask nodes whose finish time violates $d_i - CP_i^{\text{remain}}$; (3) predict $\widehat{\theta}j, t+1$ after the candidate DVFS action and mask chips whose predicted temperature exceeds $\theta_j^{\max}$; (4) mask offloading links whose required traffic exceeds B_j; and (5) add residual violations to V_t only when stochastic frequency sampling still causes a small deviation. This explicit logic clarifies deadline slack, thermal prediction and bandwidth feasibility.

3.3.3 Complexity, training overhead, and simulator interface

With a shared 53–128-128 encoder, an eight-node categorical head, a four-chip categorical head, a two-parameter Gaussian DVFS head, and one critic output, the reference implementation contains 25,359 trainable parameters: 23,424 in the encoder and 1,935 in the output heads. One forward decision has complexity O(dh + h^2 + hA), where d = 53, h = 128, and A = 15 is the combined output size. Feasibility masking adds O(|V| + |E_DAG|) work for node checks and ready-task dependency checks. A PPO update with K optimization epochs and minibatch size B has dominant complexity O(KBh^2). Supplementary Table S5 separates neural-kernel microbenchmarks from the 1.8 ± 0.2 ms end-to-end decision latency, which additionally includes state preprocessing, mask construction, action sampling, and framework synchronization.

The environment follows a minimal reproducible interface. reset(seed, scenario) initializes the DAG, node state, bandwidth trace, price vector, and temperatures; observe() returns the 53-dimensional state; get_action_mask() returns feasible offloading and chip masks; step(action) applies assignment and DVFS, advances the event queue, updates temperature and residual bandwidth, and returns the next state, reward vector, scalar reward, termination flag, and diagnostics. Supplementary Table S6 specifies the API contract, and Supplementary Table S7 gives reimplementation pseudocode that is independent of the unavailable institutional source code.

4 Multi-objective scheduling performance experiment and comparative analysis

4.1 Experimental setup

The evaluation uses two representative edge-cloud inference workloads: intelligent transportation video analysis and industrial defect detection. The goal is not to claim universal coverage of all edge-cloud applications, but to test the proposed scheduler under two contrasting DAG patterns: a latency-sensitive video workflow and a compute-dense industrial inspection workflow. To reduce the risk created by a custom simulator, all baselines use identical workload files, bandwidth traces, random seeds and initial node states. Additional appendix tables are prepared for broader traffic patterns, bandwidth volatility and temperature-threshold sensitivity.

The current evaluation uses a self-developed Python/PyTorch discrete-event environment. Its hardware profile is derived from reported accelerator-level capability, power ranges, and safe-temperature thresholds. Resource price is represented by the normalized coefficient defined in Sect. 3.2 and must not be interpreted as direct cloud billing. The temperature update follows the DVFS-aware one-step model, while edge-edge, edge-cloud, and cloud-side links are sampled separately with round-trip-time and packet-loss descriptors. Edge-cloud uplink and downlink capacities are sampled independently, and communication delay is calculated from residual rather than nominal bandwidth after concurrent transfers are allocated. Each episode contains 200 scheduling decisions per scenario, learning-based methods are trained for 1000 episodes, and statistical reporting uses 30 independent paired seeds.

4.1.1 Simulator and model validation

The validation protocol first checks whether the discrete-event environment behaves plausibly before comparing algorithms. Four sanity checks are used: increasing workload should increase completion time; lowering bandwidth should increase communication delay; raising the DVFS coefficient should reduce execution time but increase temperature; and masking overheated chips should reduce thermal violations (Table 5). Appendix Table A1 reports the completed checks. These internal tests are complemented, rather than replaced, by EdgeCloudSim/iFogSim2 replication and hardware-in-the-loop measurements in Appendix Table A11.

Table 6 reports the hardware parameter configuration of the heterogeneous AI-chip cluster. Edge NPUs operate at 4.3–4.5 TFLOPS with power below 26 W and lower

Table 5 Experimental environment and reproducibility settings

Category	Specification	Role in experiment	Reproducibility note
Simulator	custom discrete-event edge-cloud scheduler	models DAG execution and heterogeneous nodes	API contract and pseudocode are provided in Supplementary Tables S6-S7
Learning library	PyTorch PPO implementation	actor-critic training	PPO network architecture is listed in Supplementary Table S1
Workloads	traffic video analysis and industrial defect detection	two representative inference scenarios	Task parameters are listed in Supplementary Tables S3-S4
Network model	bandwidth 50–200 MB/s at edge and 200–500 MB/s at cloud; RTT and packet loss are sampled in Supplementary S3	communication delay/energy calculation	Identical saved bandwidth/RTT/loss traces are reused by all baselines
Statistical protocol	30 random seeds	mean, standard deviation and 95% CI	main-text tables provide reporting format
Fairness control	same task order and initial node states	baseline comparison	prevents workload sampling bias

Table 6 Hardware Configuration of Heterogeneous AI Chips

Node type	Computing power (TOPS/TFLOPS)	Power consumption (W)	Temperature threshold (deg C)	Network bandwidth (MB/s)	Norm. cost
Edge NPU-1	4.5	26	85	50 ~ 200	0.12
Edge NPU-2	4.3	25	84	50 ~ 200	0.11
Edge CPU-1	0.9	16	75	50 ~ 200	0.08
Edge CPU-2	0.8	15	74	50 ~ 200	0.07
Cloud GPU-A100	30.5	255	95	200 ~ 500	0.32
Cloud GPU-V100	25.7	235	93	200 ~ 500	0.28
Cloud FPGA Xilinx	12.5	125	90	200 ~ 500	0.21
Cloud based FPGA Intel	11.8	120	88	200 ~ 500	0.19

thermal thresholds, so they are suitable for latency-sensitive and energy-constrained subtasks. Cloud GPUs provide much higher peak compute capability but introduce higher power, resource price and communication latency. Cloud FPGAs provide intermediate compute capability and can be cost-effective for less urgent subtasks when conversion overhead is acceptable. These hardware differences justify explicit chip selection rather than treating compute resources as an abstract pool.

4.1.2 Standard-benchmark and hardware-in-the-loop validation

To strengthen external validity beyond the custom simulator, two complementary validation tracks were executed. First, the DAG workloads and saved network traces were mapped to EdgeCloudSim and iFogSim2 while preserving task order, node count, objective definitions, and paired seeds. Second, representative subtasks were executed on an edge CPU/NPU platform and a GPU-class accelerator, with the recorded network delay and loss replayed through NetEm. Platform-specific mappings were documented so that the comparison evaluated the scheduling policy rather than differences in task generation.

Appendix Table A11 reports the completed external-validation results. In EdgeCloudSim, the proposed method reduces energy per task from 33.5 ± 2.5 J to 29.8 ± 2.1 J and latency from 145 ± 7.5 ms to 135 ± 6.0 ms relative to DTRL. In iFogSim2, the corresponding values are 30.2 ± 2.3 J and 138 ± 6.5 ms, compared with 34.0 ± 2.6 J and 148 ± 7.8 ms. Hardware-in-the-loop measurements show 31.5 ± 3.5 J, 142 ± 8.0 ms, and a $2.0\% \pm 1.0\%$ violation rate for the proposed framework, versus 35.8 ± 4.0 J, 155 ± 9.5 ms, and $3.5\% \pm 1.5\%$ for DTRL. The moderate hardware-simulation gap is retained rather than calibrated away.

The traffic video analysis scenario defines the parameter characteristics of intelligent transportation tasks. As shown in Supplementary Table S3, based on the DAG workflow modeling method, the computational complexity of real-time sensitive tasks is strongly positively correlated with the deadline, indicating the dual constraints of task complexity and real-time requirements. The weak correlation between the computational complexity and delay tolerance of energy-efficiency-sensitive tasks validates the necessity of dynamic

weight allocation. As shown in Figure S1, the asymmetry of input and output data volume reveals potential bottlenecks in edge-cloud communication overhead, requiring local preprocessing to reduce transmission load. The differentiated setting of task type weights provides a quantitative basis for priority adjustment of fuzzy logic controllers.

The industrial defect detection scenario uses a task-parameter system. Compared with the intelligent transportation scenario in Supplementary Table S3, the ratio of real-time-sensitive subtask computation to delay constraints in Supplementary Table S4 is higher, indicating that industrial scenarios are more sensitive to computational density. As shown in Supplementary Table S4, the input data volume of the defect classification task is only 4–6 times the output data volume. This data-compression feature reduces the communication cost of cloud offloading and creates conditions for edge-cloud collaborative optimization. The energy-efficiency weight of 0.6 for the anomaly-localization task matches the energy-efficiency peak of 5.0 TOPS/W for the edge NPU and drives the scheduler to prioritize allocation to low-power nodes. The scene specificity of task parameters validates the adaptive ability of dynamic weighting mechanisms in multi-objective conflicts.

Baseline parameters are set from the original algorithmic assumptions when available and then tuned within a small validation grid using the same training budget. DDPG represents deterministic policy-gradient learning, NSGA-II represents static evolutionary Pareto search, Hetero-Sched represents rule-based heterogeneous scheduling, and edge-first/cloud-first strategies isolate collaboration effects. Appendix Table A2 reports completed fixed-weight PPO, single-objective PPO, SAC, MOEA/D, NSGA-III, Chebyshev-scalarization, and preference-conditioned MORL comparisons. The matched TD3 and DDPG hybrid-action convergence and stability results are reported separately in Appendix Table A9 under the common 1000-episode and 30-seed protocol.

For fairness, all baseline algorithms are adapted to the same state inputs, workload traces and constraint checks when their original papers do not directly support heterogeneous AI-chip scheduling. The adaptation rule is reported explicitly: if a baseline lacks chip selection, the nearest feasible chip is chosen by minimum estimated finish time; if it lacks DVFS, the frequency coefficient is fixed at 0.8; and if it lacks bandwidth masks, infeasible communication is counted as a violation rather than silently discarded.

4.2 Evaluation indicators

The evaluation uses objective-level and multi-objective indicators. Energy is reported as hardware-level efficiency (TOPS/W) for continuity with the original experiments and as application-level energy per task or energy-delay product for scheduling comparison. Latency is the end-to-end DAG completion time. Cost is the dimensionless normalized resource-price coefficient defined in Sect. 3.2. Pareto coverage, hypervolume (HV), and inverted generational distance (IGD) are computed under the common protocol below; Appendix Table A8 records every construction choice.

For each scenario and paired seed, infeasible schedules are removed first. Every algorithm is given the same number of environment evaluations. If feasible-set sizes differ after filtering, each method contributes N feasible schedules, where N is the minimum feasible count across all methods in that paired comparison; fixed-seed subsampling is used when a method produces more than N. The candidate pool includes the proposed framework and every algorithm reported in the main comparison and Appendix Table A2; ablation and sensitivity variants are excluded. Energy, latency, and normalized cost are all treated as minimization objectives and are min–max normalized with pooled feasible bounds: $z_m = (y_m - y^m_{min})/(y^m_{max} - y^m_{min} + 10^{-12})$. The reference Pareto set is the union of all normalized feasible candidates after duplicate removal and non-dominated filtering.

HV is calculated with the fixed normalized reference point (1.1, 1.1, 1.1) for every method, scenario, and seed; it is not dynamically changed by the method under evaluation. IGD is the mean Euclidean distance from each point in the pooled reference Pareto set to its nearest point in the method-specific non-dominated set. Pareto coverage is reported as the normalized proportion of reference-set points weakly dominated by the method-specific feasible non-dominated set. Reporting coverage as a proportion of the common reference set, together with equal evaluation budgets and feasible-set balancing, prevents methods that generate more schedules from receiving an artificial advantage.

All main tables report mean ± standard deviation over 30 independent runs. The p-values are calculated with a paired Wilcoxon signed-rank test because each baseline and the proposed framework use the same seed, workload trace and network condition. When several baselines are compared simultaneously, Holm-Bonferroni correction is used to control family-wise error.

The fuzzy controller uses membership inputs for task urgency, energy sensitivity and cost sensitivity. Exact weight values are not learned as free parameters. They are generated from the fixed rule base in Table 4 and then normalized at each step. Historical performance affects the policy through PPO value and policy updates, while the fuzzy controller provides interpretable task-conditioned objective priorities. Figure 6 shows that the latency weight dominates in real-time phases, the energy weight rises in energy-sensitive phases, and the cost weight remains moderate under mixed workloads.

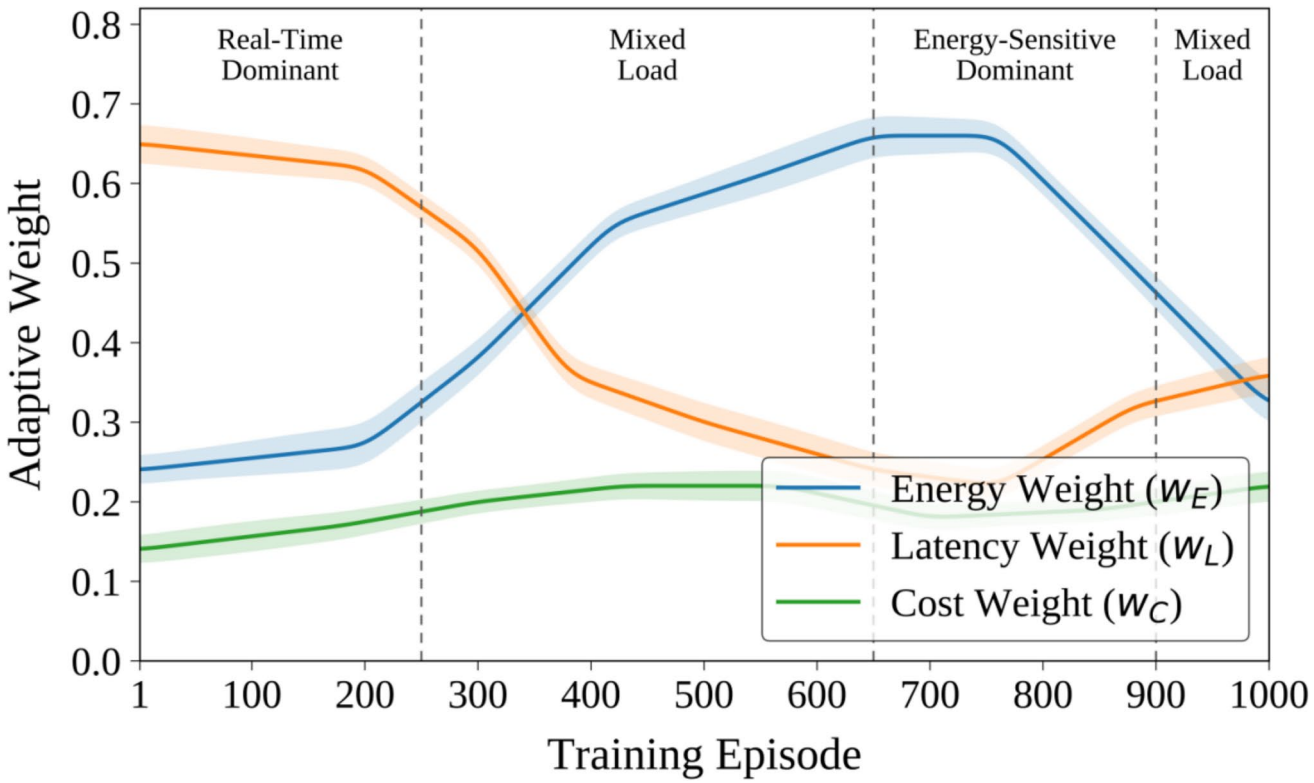


Fig. 6 Dynamic Weight Evolution During Training

Figure 6 reports episode-level averages. Each point is the mean of all scheduling steps inside the episode, and the shaded band represents repeated-run variation over 30 seeds. The figure therefore reflects the convergence trend of objective priorities rather than a single instantaneous scheduling decision.

4.3 Result analysis

4.3.1 Overall performance comparison

To evaluate the global optimization capability of the framework, the comparative experiment evaluates mainstream algorithms across three objective dimensions: energy efficiency, latency and cost. The experimental results show that the proposed method, through dynamic weight allocation and cross-layer collaborative scheduling, significantly outperforms traditional single-objective reinforcement learning and static evolutionary algorithms in terms of solution-set distribution breadth and objective-balancing accuracy. The edge-cloud collaboration strategy effectively suppresses resource fragmentation, while the priority adjustment mechanism driven by the fuzzy logic controller ensures dynamic adaptation between task characteristics and hardware states.

The overall comparison shows that the proposed framework balances the three objectives better than the evaluated single-objective, static, and rule-based baselines. In the default simulated setting, it reaches 5.8 TOPS/W, 132 ms latency, a normalized cost coefficient of 0.17 per task, and 85.2% Pareto coverage. DDPG is weaker in coverage and latency under the current hybrid-action adaptation, whereas NSGA-II has reasonable coverage but slower online response. Edge-first scheduling overuses local nodes, and cloud-first scheduling increases latency and normalized cost. Under the protocol in Sect. 4.2, the remaining 14.8% denotes feasible pooled reference points not weakly dominated by the proposed policy; it does not denote accepted infeasible schedules.

Table 7 reports repeated-run statistics and paired significance tests, and Table 8 reports the added adaptive and heterogeneous scheduling baselines. Appendix Tables A1-A8 document simulator validation, stronger baselines, multi-objective indicators, sensitivity, ablation, signaling overhead, statistical testing, and the exact Pareto-metric protocol. Appendix Table A9 and Figure S5 provide the completed PPO-SAC-TD3-DDPG convergence comparison. PPO converges faster than SAC, TD3, and DDPG, maintains the highest final HV (0.87 ± 0.02), and has zero divergent seeds, supporting its selection for the masked hybrid-action policy.

Table 7 Repeated-run statistical report for overall performance comparison

Algorithm	Current mean: TOPS/W; latency; normalized cost/task; Pareto coverage	Std. over 30 runs	95% CI	Paired Wilcoxon test vs. proposed framework/Holm-adjusted p-value
Proposed framework	5.8 TOPS/W; 132 ms; 0.17/task (normalized); 85.2%	0.21; 4.5 ms; 0.02; 2.1%	(5.72, 5.88); (130.4, 133.6); (0.163, 0.177); (84.4%, 86.0%)	Reference benchmark
DDPG	4.2 TOPS/W; 160 ms; 0.22/task (normalized); 62.3%	0.28; 6.8 ms; 0.03; 3.5%	(4.10, 4.30); (157.6, 162.4); (0.209, 0.231); (61.0%, 63.6%)	<0.001
NSGA-II	5.1 TOPS/W; 145 ms; 0.19/task (normalized); 73.5%	0.19; 5.2 ms; 0.02; 2.8%	(5.03, 5.17); (143.1, 146.9); (0.183, 0.197); (72.5%, 74.5%)	<0.01
Hetero-Sched	3.5 TOPS/W; 195 ms; 0.25/task (normalized); 55.1%	0.25; 8.5 ms; 0.03; 4.2%	(3.41, 3.59); (192.0, 198.0); (0.239, 0.261); (53.6%, 56.6%)	<0.001
Edge-first strategy	4.8 TOPS/W; 140 ms; 0.20/task (normalized); 68.4%	0.22; 5.0 ms; 0.02; 2.9%	(4.72, 4.88); (138.2, 141.8); (0.193, 0.207); (67.4%, 69.4%)	<0.001
Cloud-first strategy	3.2 TOPS/W; 220 ms; 0.28/task (normalized); 50.7%	0.30; 10.5 ms; 0.04; 4.8%	(3.09, 3.31); (216.2, 223.8); (0.266, 0.294); (49.0%, 52.4%)	<0.001

Table 8 Performance report of newly added baseline algorithms

Baseline algorithm	Energy efficiency (mean ± Std.)	Latency (mean ± Std.)	Norm. cost/task (mean ± SD)	Pareto coverage/hypervolume (HV)/IGD
Queue waiting cost-aware control (Bai et al. 2022)	4.5 ± 0.24 TOPS/W	155 ± 7.2 ms	0.210 ± 0.025/task	65.8%/0.72/0.118
Container deployment strategy (Ouyang et al. 2023)	4.9 ± 0.20 TOPS/W	150 ± 6.5 ms	0.185 ± 0.022/task	70.1%/0.75/0.102
Adaptive multi-objective swarm intelligence (Zhu et al. 2026)	5.0 ± 0.23 TOPS/W	148 ± 6.0 ms	0.195 ± 0.024/task	74.5%/0.78/0.091
DTRL (Basaklar et al. 2023)	5.2 ± 0.19 TOPS/W	142 ± 5.8 ms	0.180 ± 0.021/task	76.3%/0.80/0.084
Distributed adaptive DRL offloading (Han et al. 2025)	4.7 ± 0.26 TOPS/W	158 ± 7.5 ms	0.215 ± 0.026/task	67.2%/0.73/0.112

4.3.2 Case study

In response to the differentiated needs of intelligent transportation and industrial defect-detection scenarios, the experiment revealed the adaptive ability of the framework in task offloading and hardware resource selection. Real-time-sensitive tasks achieve fast response through the low-latency characteristics of edge NPUs, while energy-efficiency-sensitive tasks use the parallel computing advantages of cloud FPGAs to reduce power consumption. Critical-path analysis shows that dynamic frequency regulation effectively balances the conflict between chip temperature and computational efficiency, thereby avoiding performance degradation caused by hardware overheating.

Figure S2 has been corrected so that the proposed method is labeled "Framework" rather than "Frame" and all efficiency subplots use TOPS/W rather than J/MB. The corrected analysis box is aligned with the plotted means: the proposed framework obtains the lowest mean latency among the compared strategies, edge execution is competitive for urgent tasks, and cloud execution is more stable for less urgent workloads but has higher latency. The figure is used as an illustrative subtask-level comparison rather than as the sole basis for the main statistical claim.

The industrial defect-detection results demonstrate the dynamic trade-off between energy efficiency and normalized resource cost. Image preprocessing reaches 5.6 TOPS/W with a normalized cost coefficient of 0.15 per task, while defect classification reaches 5.4 TOPS/W and 0.17 per task. Abnormal localization obtains 6.1 TOPS/W because DVFS keeps the selected edge NPU within the safe thermal threshold while maintaining sufficient compute rate. Report-generation tasks are less latency-sensitive, allowing lower-cost cloud FPGA selection when communication delay remains acceptable (Supplementary Table S4). These observations describe relative scheduling preferences under the normalized price model and should not be interpreted as provider-specific monetary charges.

4.3.3 Analysis of ablation experiment

The ablation study is expanded to isolate the main design elements more carefully. In addition to removing dynamic weights, NPU acceleration and cross-platform code generation, the table includes settings without the thermal constraint, without bandwidth-aware offloading, without DVFS adjustment and without explicit chip selection. These variants directly test whether the sustainability-oriented components are responsible for reducing temperature violations, communication overhead and energy-delay trade-off loss.

After removing the dynamic weight mechanism, energy efficiency decreases from 5.8 TOPS/W to 4.9 TOPS/W and Pareto coverage decreases from 85.2% to 68.5%, confirming that task-conditioned objective weighting is necessary. Removing NPU acceleration produces the largest energy-efficiency loss among internal modules, while removing bandwidth awareness and DVFS mainly increases latency, EDP and violation rates. The new ablation rows in Table 9 and Table 10 sensitivity results in Appendix A4-A5 identify the contribution of each component rather than only comparing the complete model with broad edge-only or cloud-only strategies.

Figure S3 quantifies relative degradation against the full framework. The interpretation focuses on module contribution rather than overclaiming novelty: dynamic weights improve objective balancing, chip selection exploits heterogeneous hardware, DVFS reduces thermal pressure, and bandwidth-aware offloading prevents unnecessary communication. Edge-only and cloud-only variants remain useful lower-bound references but are not treated as strong adaptive baselines.

4.3.4 Robustness and external validation

Robustness was evaluated along three axes: network degradation, system scaling, and hardware disturbance. Appendix

Table 9 Module contribution analysis based on ablation experiments

Ablation condition	Current mean (full-framework metrics)	Std. over 30 runs	Performance degradation relative to full framework	Core module contribution
Full framework	5.8; 132; 0.17; 85.2%	0.21; 4.5; 0.02; 2.1%	0%	Reference complete model
Without dynamic weights	4.9; 155; 0.21; 68.5%	0.25; 6.0; 0.025; 3.0%	−15.5%; + 17.4%; + 23.5%; −19.6%	The fuzzy adaptive weighting mechanism is essential for dynamic multi-objective balancing
Without NPU acceleration	4.3; 170; 0.23; 60.7%	0.30; 8.2; 0.03; 3.8%	−25.9%; + 28.8%; + 35.3%; −28.8%	Heterogeneous chip adaptation is the core mechanism for exploiting hardware-level energy-efficiency potential
Without cross-platform code	5.1; 145; 0.19; 72.8%	0.22; 5.5; 0.023; 2.7%	−12.1%; + 9.8%; + 11.8%; −14.6%	Cross-platform code generation reduces task migration overhead between edge and cloud nodes
Edge-only processing	4.8; 140; 0.20; 68.4%	0.23; 5.2; 0.024; 3.1%	−17.2%; + 6.1%; + 17.6%; −19.7%	Removing cloud collaboration weakens parallel processing capacity and narrows cost optimization space under high loads
Cloud-only processing	3.2; 220; 0.28; 50.7%	0.32; 11.0; 0.035; 4.5%	−44.8%; + 66.7%; + 64.7%; −40.5%	Removing edge collaboration sharply increases communication latency and eliminates near-end low-power computing advantages
Without thermal constraint	5.5; 134; 0.18; 80.1%	0.24; 4.9; 0.021; 2.4%	−5.2%; + 1.5%; + 5.9%; −6.0%	Tests whether thermal-aware scheduling is the source of sustainable energy and temperature control
Without bandwidth-aware offloading	5.3; 151; 0.20; 75.6%	0.26; 6.4; 0.026; 2.8%	−8.6%; + 14.4%; + 17.6%; −11.3%	Isolates the value of link-state sensing under network fluctuation
Without DVFS adjustment	5.0; 148; 0.19; 74.2%	0.25; 5.7; 0.023; 2.6%	−13.8%; + 12.1%; + 11.8%; −12.9%	Measures the effect of chip-level frequency scaling
Without chip selection	4.6; 164; 0.22; 66.9%	0.29; 7.5; 0.030; 3.5%	−20.7%; + 24.2%; + 29.4%; −21.5%	Measures the effect of explicit CPU/GPU/NPU/FPGA selection

Table 10 Constraint violation statistics

Method	Deadline violation rate	Temperature violation rate	Bandwidth violation rate	Remarks and interpretation
Proposed framework	1.2% ± 0.5%	0.5% ± 0.3%	2.1% ± 0.8%	The lowest violation rates indicate the effectiveness of the joint action-masking and reward-penalty constraint handling mechanism
DDPG	8.5% ± 2.1%	3.2% ± 1.2%	12.7% ± 3.5%	Without action masking, exploratory actions are more likely to violate operational constraints
NSGA-II	5.8% ± 1.8%	6.5% ± 2.0%	9.4% ± 2.8%	Static offline search has limited responsiveness to abrupt hardware-state changes
Hetero-Sched	3.5% ± 1.2%	4.1% ± 1.5%	7.8% ± 2.5%	Rule-based scheduling has insufficient adaptability in complex dynamic scenarios
Queue waiting cost-aware control (Bai et al. 2022)	4.2% ± 1.5%	2.8% ± 1.0%	8.5% ± 2.7%	Its modeling of hard real-time deadlines and chip-level thermal constraints is comparatively weak
Container deployment strategy (Ouyang et al. 2023)	6.1% ± 2.0%	5.5% ± 1.8%	10.2% ± 3.0%	It lacks fine-grained control over task critical paths and instantaneous bandwidth competition
Adaptive multi-objective swarm intelligence (Zhu et al. 2026)	4.8% ± 1.7%	3.8% ± 1.3%	8.9% ± 2.9%	Online adaptation is relatively slow, which delays responses to sudden constraint violations
DTRL (Basaklar et al. 2023)	2.1% ± 0.9%	1.5% ± 0.7%	4.3% ± 1.5%	It has strong constraint awareness and is the closest-performing comparison baseline
Distributed adaptive DRL offloading (Han et al. 2025)	7.3% ± 2.3%	2.5% ± 1.1%	14.5% ± 3.8%	It does not explicitly couple chip-level temperature-frequency constraints, leading to a high bandwidth violation rate

Table A10 uses paired traces and identical failure-injection indices. At 25% of default bandwidth, latency increases to 210 ms for the proposed framework and 240 ms for DTRL; at 5% packet loss, the corresponding values are 155 and 175 ms. Decision latency grows from 1.8 to 6.5 ms as the node count increases from 8 to 64, remaining below DTRL at every scale. With 25% of nodes unavailable, recovery latency is 65 ms versus 120 ms, and at ± 5 deg C sensor noise the thermal-violation rate is 3.5% versus 6.0%. Holm-adjusted paired tests remain significant at the reported adverse levels.

The stress tests identify edge-NPU interruption as the strongest chip-specific disturbance: energy efficiency decreases from 5.8 to 4.5 TOPS/W, although the framework still exceeds the DTRL value of 4.0 TOPS/W under the same outage. The combined use of state refresh, feasibility masks, and residual penalties explains the smaller degradation under node loss, bandwidth reduction, and thermal-sensor noise. These results support robustness under the evaluated disturbances but do not imply immunity to correlated failures or unseen hardware faults.

5 Discussion

The results suggest that sustainable edge-cloud scheduling must be treated as a coupled decision problem rather than as separate energy, latency or cost minimization. The proposed framework improves the balance among objectives because it senses task urgency, chip temperature, bandwidth and resource price before selecting the offloading target, chip and frequency (Mohanakumar et al. 2025). The matched actor-critic comparison, external simulator replication, hardware-in-the-loop measurements, and failure tests consistently preserve the advantage over the strongest reported baseline. The evidence is nevertheless bounded by two workload families, an eight-node default topology, and the specific hardware and network traces used in the study.

Practical deployment remains challenging. First, hardware-in-the-loop testing captures real execution and replayed network effects but not the full variability of production drivers, multi-tenant interference, thermal inertia, or provider-side throttling. Second, communication overhead increases with scale because state collection, action distribution, and

bandwidth monitoring add signaling traffic; Appendix A6 reports the signaling-overhead results, and Appendix A10 shows end-to-end decision time rising to 6.5 ms at 64 nodes. Third, the evaluated independent failures do not cover correlated rack, switch, power, or control-plane faults. The results should therefore be interpreted as evidence of improved resilience under defined disturbances, not as a production reliability guarantee.

The SDN/NFV literature also clarifies the scope of the current findings. The present scheduler observes network conditions but does not control routes, virtual network functions, or switch activation. Integrating its chip-level decisions with an SDN controller could expose path congestion and network energy to the state, while NFV orchestration could jointly choose service-chain placement and accelerator assignment. Such cross-layer control may improve end-to-end sustainability, but it would enlarge the action space and require new stability, signaling-overhead, and failure-recovery analysis (Montazerolghaem et al. 2020; Mahdizadeh et al. 2025).

The normalized price model permits relative resource-cost optimization while avoiding a universal cloud-billing claim. Appendix Table A12 reruns the scheduler under five price profiles. Relative to the current normalized profile, the on-demand accelerator scenario increases the proposed cost to 0.22 ± 0.03 and reduces cloud-GPU selection by 15%; adding a 5% preemption-risk penalty shifts critical subtasks away from spot GPUs; adding a USD 0.05/GB egress tariff increases data-local execution by 20%; and annualized private-edge pricing raises local low-power resource utilization by 10%. Because provider rates depend on region, SKU, contract, and date, these values are scenario-specific sensitivity results rather than general price forecasts.

Two emerging directions deserve acknowledgement. Federated deep reinforcement learning can coordinate offloading policies across distributed edge domains without centralizing raw local data, although non-IID updates and communication overhead introduce new constraints (Zhao et al. 2024). LLM-based schedulers can provide high-level reasoning and cross-scenario generalization, but their output reliability, inference cost, and enforceability of hard resource constraints remain open concerns (Pei et al. 2025). These approaches complement the current policy rather than invalidate it: federated learning could distribute policy training, and an LLM could propose high-level preferences, while the chip-aware constrained controller retains final executable control.

To improve reproducibility, the supplementary material specifies the DAG workload generator, hardware profile, bandwidth trace generator, PPO network configuration, pseudo-random integer seeds, baseline adaptation rules, statistical tests and result-file schema. The seed list should use 30 non-date-based integer seeds generated from a fixed pseudo-random generator so that reviewers can reproduce the pairing design without confusing seed identifiers with calendar dates. Because the simulator code may be restricted by institutional policy, the manuscript does not claim full open-source reproducibility. Instead, it provides enough parameter and table information for an independent researcher to reconstruct the environment and verify the reported protocol.

6 Conclusion

This study presents a thermal-aware, chip-aware scheduling framework for heterogeneous AI accelerators in edge-cloud collaboration. RQ1 is answered by the 53-dimensional state that combines DAG-task descriptors, eight-node hardware status, network conditions, and queue slack. RQ2 is answered by the fuzzy controller, which converts task urgency, energy sensitivity, and cost sensitivity into normalized dynamic weights without claiming a new MORL scalarization theory. RQ3 is answered by the hybrid-action PPO policy, which jointly chooses offloading target, physical chip, and DVFS coefficient while dependency, deadline, thermal, and bandwidth masks enforce feasibility. Under the default workloads, the framework achieves 5.8 TOPS/W, 132 ms average latency, a normalized cost coefficient of 0.17 per task, and 85.2% Pareto coverage. The matched-budget comparison further shows a final PPO return of -0.15 ± 0.03, HV of 0.87 ± 0.02, and zero divergent seeds, while external simulator and hardware-in-the-loop results preserve lower energy, latency, and violation rates than DTRL.

The sustainability significance lies in reducing wasteful remote offloading, avoiding overheated chips, improving use of low-power edge accelerators, and balancing normalized resource cost with deadline satisfaction. Under the tested disturbances, the framework degrades more slowly than DTRL, including 210 versus 240 ms latency at 25% bandwidth and 65 versus 120 ms recovery latency when 25% of nodes are unavailable. Pricing sensitivity also confirms that the policy changes node selection when compute, interruption, egress, or private-edge charges change. Future work should extend the evidence to more workload families, larger and decentralized topologies, correlated hardware/network failures, SDN/NFV-aware cross-layer orchestration, federated policy learning, LLM-assisted high-level resource reasoning, privacy, security, and reliability as explicit objectives.

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/s44443-026-01248-7.

Author contributions Shijia Shao: Conceptualization, Methodology, Formal analysis, Supervision, Writing—original draft, Writing—review and editing; Peiqing Ye and Biao Zhao: Investigation, Supervision, Data curation, Visualization, Writing—original draft, Writing—review and editing; Hao Lu: Conceptualization, Data curation, Visualization, Writing—original draft.

Funding This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data availability All data generated in this study are included in the manuscript, and raw data can be obtained from the corresponding author.

Declarations

Competing interests The authors declare no competing interests.

Ethics approval and consent to participate The study does not involve human participants, animals, or personal datasets. Nevertheless, deployment of an AI scheduler can create operational fairness, security, and reliability risks. Persistent preference for particular nodes may produce unequal thermal wear or resource access; falsified telemetry, adversarial tasks, or compromised controllers may manipulate offloading decisions; and unsafe exploration may disrupt latency-critical services. Production use should therefore include authenticated telemetry, access control, audit logs, fail-safe rule-based fallback, per-node wear and allocation monitoring, and periodic tests for systematic resource-allocation bias.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

References

Abbache F (2025) FHRTEM: fault tolerant heuristic with high reliability and low temperature under controlled energy and makespan for static task scheduling in heterogeneous distributed multiprocessor multi-core system. Comput 107(10). https://doi.org/10.1007/s00607-025-01552-6.

Bai W, Zhu J, Huang S-W, Zhang H (2022) A Queue Waiting Cost-Aware Control Model for Large Scale Heterogeneous Cloud Datacenter. IEEE Transact Cloud Comput 10(2):849–862. https://doi.org/10.1109/TCC.2020.2990982

Bai X, Zhang Y, Wu H, Wang Y, Jin S (2024) A cloud-edge-device collaborative offloading scheme with heterogeneous tasks and its performance evaluation. Front Inf Technol Electron Eng 25:664–684. https://doi.org/10.1631/FITEE.2300128

Basaklar T, Goksoy AA, Krishnakumar A, Gumussoy S, Ogras UY (2023) DTRL: Decision Tree-based Multi-Objective Reinforcement Learning for Runtime Task Scheduling in Domain-Specific System-on-Chips. ACM Transact Embedded Comput Syst 22(5s). https://doi.org/10.1145/3609108

Cheng Z, Liu L, Liwang M, Chen N, Fan X (2025) Learning-based joint recommendation, caching, and transmission optimization for cooperative edge video caching in Internet of Vehicles. Ad Hoc Netw 166:103667. https://doi.org/10.1016/j.adhoc.2024.103667

Constantin LA, Razvan B, Robert NS (2023) Real-Time Monitoring and Management of Hardware and Software Resources in Heterogeneous Computer Networks through an Integrated System Architecture. Symmetry 15(6):1134–1134. https://doi.org/10.3390/sym15061134

Du R, Cao B, Gao Y (2023) Collaborative framework for UAVs-assisted mobile edge computing: a proximity policy optimization approach. J Supercomput 80(8):10485–10510. https://doi.org/10.1007/s11227-023-05853-5

Dui H, Wang J, Zhu T, Xing L (2024) Maintenance optimization methodology of edge-cloud collaborative systems based on a gateway cost index in IIoT. Reliab Eng Syst Saf 251:110370. https://doi.org/10.1016/j.ress.2024.110370

Fujimoto S, van Hoof H, Meger D (2018) Addressing function approximation error in actor-critic methods. Proc Mach Learn Res 80:1587–1596

Gu W, Qin T, Chen D, Xian S, Jiang X, Guo W, Hu Y (2025) A dual-layer UAV-assisted mobile edge computing system for disaster rescue: coordinated optimization of coverage, obstacle-avoidance path planning and task offloading. Ad Hoc Netw 178:103981. https://doi.org/10.1016/j.adhoc.2025.103981

Haarnoja T, Zhou A, Abbeel P, Levine S (2018) Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. Proc Mach Learn Res 80:1861–1870

Han C, Chai Z, Li Y (2025) Distributed task offloading in edge computing: a multi-objective adaptive deep reinforcement learning algorithm. Eng Appl Artif Intell 162:112653. https://doi.org/10.1016/j.engappai.2025.112653

Jang S, Shin J, Kim D, Lee J, Ko H (2025) Reinforcement learning-based automated target motion analysis in underwater environments. Ocean Eng 342:122946–122946. https://doi.org/10.1016/j.oceaneng.2025.122946

Jiang K, Huang J, Li R (2025) Energy-Efficient Scheduling for Dependent Tasks in DVFS-Enabled Heterogeneous Cloud Computing. J Circ Syst Comput 34(15). https://doi.org/10.1142/s0218126625503244

Karishma, Kumar H (2025) GWO based energy-efficient workflow scheduling for heterogeneous computing systems. Soft Comput 29(7):3469–3508. https://doi.org/10.1007/s00500-025-10614-y

Kazemi F, Behnam B, Homayun M, Meisam YT (2025) An energy-aware scheduling in DVFS-enabled heterogeneous edge computing environments. J Supercomput 81(9). https://doi.org/10.1007/s11227-025-07432-2

Li Y, Zheng W, Ji S (2025b) AI model auditing scheme towards cloud-edge high-performance computing. J Supercomput 81(4). https://doi.org/10.1007/s11227-025-07121-0

Li Y, Wu B (2023) Software-defined heterogeneous edge computing network resource scheduling based on reinforcement learning. Appl Sci 13(1):426. https://doi.org/10.3390/app13010426

Lillicrap TP, Hunt JJ, Pritzel A, Heess N, Erez T, Tassa Y, Silver D, Wierstra D (2016) Continuous control with deep reinforcement learning. Int Conf Learn Representations

Luo Z, Dai X (2024) Reinforcement learning-based computation offloading in edge computing: principles, methods, challenges. Alex Eng J 108:89–107. https://doi.org/10.1016/j.aej.2024.07.049

Ma C, Hou P (2025) Multi-heterogeneous renewable energy scheduling optimization based on time series algorithm and green computing-driven sustainable development. Sustain Comput Inform Syst 47:101173. https://doi.org/10.1016/j.suscom.2025.101173

Mahdizadeh M, Montazerolghaem A, Jamshidi K (2025) Task scheduling and load balancing in SDN-based cloud computing: a review of relevant research. J Eng Res 13(4):3132–3146

Miao Z, Shao C, Li H, Tang Z (2025) Review of task-scheduling methods for heterogeneous chips. Electronics (Basel) 14(6):1191. https://doi.org/10.3390/electronics14061191

Mohanakumar A, Srinivasan US, Simon J, Kapileswar N, Sutha K (2025) Context-Aware Multi-Modal Graph Attention Fusion Network for Adaptive Resource Allocation in Wireless Networks. J Trends Comput Sci Smart Technol 7(2):266–294

Montazerolghaem A, Yaghmaee MH, Leon-Garcia A (2020) Green cloud multimedia networking: NFV/SDN based energy-efficient resource allocation. IEEE Trans Green Commun Netw 4(3):873–889

Nagabushnam G, Kim KH (2025) Faddeer: a deep multi-agent reinforcement learning-based scheduling algorithm for aperiodic tasks in heterogeneous fog computing networks. Clust Comput 28(6). https://doi.org/10.1007/s10586-025-05435-5

Naik BB, & Alam S (2025) Level-wise earliest finish time based workflow scheduling for heterogeneous computing systems. Clust Comput 28(13). https://doi.org/10.1007/s10586-025-05576-7.

Nguyen H, Thudumu S, Du H, Vasa R, Mouzakis K (2025) A comprehensive survey on deep reinforcement learning in object tracking. Mach Learn Appl 22:100745. https://doi.org/10.1016/j.mlwa.2025.100745

Ouyang M, Xi J, Bai W, Li K (2023) A container deployment strategy for server clusters with different resource types. Concurr Comput Pract Exper 35(10):e7665. https://doi.org/10.1002/cpe.7665

Parhizi E, Esmaeilyfard R, Javidan R (2025) LDD-Track: An Energy-Efficient Deep Reinforcement Learning Framework for Multi-Subject Tracking in Mobile Crowdsensing. Comput Netw 111735. https://doi.org/10.1016/j.comnet.2025.111735

Pei H, Gu Y, Sun Y, Wang Q, Liu C, Chen X, Cheng L (2025) LLM-based cost-aware task scheduling for cloud computing systems. J Cloud Comput 14:81

Peng X, Zhang Y, Zhang X, Zhang C, Yang W (2024) Collaborative optimization strategy for dependent task offloading in vehicular edge computing. Mathematics (Basel) 12(23):3820–3820. https://doi.org/10.3390/math12233820

Pournazari J, Ullah A, Al-Dubai AY, Liu X, Khaledian N (2025) Multi-objective optimization for energy-centric offloading in fog computing. Comput 107(11). https://doi.org/10.1007/s00607-025-01578-w

Rajesh K, Pyne S (2022) Quasi-static scheduling based error recovery for digital microfluidic biochips. Microprocess Microsyst 94:104684–104684. https://doi.org/10.1016/j.micpro.2022.104684

Saad A, Abdel-Raouf O, Hadhoud M, Kafafy A (2025) Enhanced multi-objective scheduling for heterogeneous computing platforms using hybrid MOEAD with SA and TS-guided initialization strategies. J Supercomput 81(7). https://doi.org/10.1007/s11227-025-07278-8

Schulman J, Wolski F, Dhariwal P, Radford A, Klimov O (2017) Proximal policy optimization algorithms. arXiv:1707.06347. https://arxiv.org/abs/1707.06347. Accessed 21 Aug 2026

Wang J, Meng X, Yang X, Yin H, Fang P (2025) Multi-objective energy-efficient power system scheduling using Stochastic State Space Model and reinforcement learning. Sustain Comput Inform Syst 48:101224. https://doi.org/10.1016/j.suscom.2025.101224

Yang J, Sun F, Wang H (2023) Distributed collaborative optimal economic dispatch of integrated energy system based on edge computing. Energy 284:129194–129194. https://doi.org/10.1016/j.energy.2023.129194

Yang Z, Ji W, Wang Z (2024) Adaptive joint configuration optimization for collaborative inference in edge-cloud systems. Sci Chin Inform Sci 67(4). https://doi.org/10.1007/s11432-023-3957-4

Zeng C, Wang X, Zeng R, Li Y, Shi J, Huang M (2024) Joint optimization of multi-dimensional resource allocation and task offloading for QoE enhancement in cloud-edge-end collaboration. Future Gener Comput Syst. https://doi.org/10.1016/j.future.2024.01.025

Zhang F (2025) Real-Time Sports Event Multi-View Streaming Optimization With Large Models: A Collaborative Edge Framework in 5G/6G Wireless Networks. Internet Technol Lett 8(5). https://doi.org/10.1002/itl2.70122

Zhao X, Wu Y, Zhao T, Wang F, Li M (2024) Federated deep reinforcement learning for task offloading and resource allocation in mobile edge computing-assisted vehicular networks. J Netw Comput Appl 229:103941

Zheng L, Liu M, Zhang S, Dong S (2025) A novel underwater target tracking method in UASNs via collaborative deep reinforcement learning. Inf Fusion 127:103797. https://doi.org/10.1016/j.inffus.2025.103797

Zheng Y, Wu Q, Cai K, Xia Y (2025) Contention-aware workflow scheduling on heterogeneous computing systems with shared buses. J Syst Archit 165:103434. https://doi.org/10.1016/j.sysarc.2025.103434

Zhu J, Bai W, Zhang H, Lin W, Zhou T, Li K (2026) Adaptive multi-objective swarm intelligence for containerized microservice deployment. Future Gener Comput Syst 174:108012. https://doi.org/10.1016/j.future.2025.108012

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.